JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class TimeZone

java.lang.Object
    java.util.TimeZone

All Implemented Interfaces:

`Serializable`, `Cloneable`

Direct Known Subclasses:

`SimpleTimeZone`

```
public abstract class TimeZone
extends Object
implements Serializable, Cloneable
```

`TimeZone` represents a time zone offset, and also figures out daylight savings.

Typically, you get a `TimeZone` using `getDefault` which creates a `TimeZone` based on the time zone where the program is running. For example, for a program running in Japan, `getDefault` creates a `TimeZone` object based on Japanese Standard Time.

You can also get a `TimeZone` using `getTimeZone` along with a time zone ID. For instance, the time zone ID for the U.S. Pacific Time zone is "America/Los_Angeles". So, you can get a U.S. Pacific Time `TimeZone` object with:

```
TimeZone tz = TimeZone.getTimeZone("America/Los_Angeles");
```

You can use the `getAvailableIDs` method to iterate through all the supported time zone IDs. You can then choose a supported ID to get a `TimeZone`. If the time zone you want is not represented by one of the supported IDs, then a custom time zone ID can be specified to produce a TimeZone. The syntax of a custom time zone ID is:

```
CustomID:
        GMT Sign Hours : Minutes : Seconds
        GMT Sign Hours : Minutes
        GMT Sign Hours Minutes
        GMT Sign Hours
Sign: one of
        + -
Hours:
        Digit
        Digit Digit
Minutes:
        Digit Digit
```

```
Seconds:
        Digit Digit
Digit: one of
        0 1 2 3 4 5 6 7 8 9
```

Hours must be between 0 to 23 and *Minutes*/*Seconds* must be between 00 to 59. For example, "GMT+10" and "GMT+0010" mean ten hours and ten minutes ahead of GMT, respectively.

The format is locale independent and digits must be taken from the Basic Latin block of the Unicode standard. No daylight saving time transition schedule can be specified with a custom time zone ID. If the specified string doesn't match the syntax, `"GMT"` is used.

When creating a `TimeZone`, the specified custom time zone ID is normalized in the following syntax:

```
NormalizedCustomID:
        GMT Sign TwoDigitHours : Minutes [ColonSeconds]
Sign: one of
        + -
TwoDigitHours:
        Digit Digit
Minutes:
        Digit Digit
ColonSeconds:
        : Digit Digit
Digit: one of
        0 1 2 3 4 5 6 7 8 9
```

For example, TimeZone.getTimeZone("GMT-8").getID() returns "GMT-08:00". *ColonSeconds* part only appears if the seconds value is non-zero.

Three-letter time zone IDs

For compatibility with JDK 1.1.x, some other three-letter time zone IDs (such as "PST", "CTT", "AST") are also supported. However, **their use is deprecated** because the same abbreviation is often used for multiple time zones (for example, "CST" could be U.S. "Central Standard Time" and "China Standard Time"), and the Java platform can then only recognize one of them. The full list of deprecated three-letter time zone IDs and their mappings can be viewed at `ZoneId.SHORT_IDS`.

Since:

1.1

See Also:

Calendar, GregorianCalendar, SimpleTimeZone, Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final int	LONG	A style specifier for getDisplayName() indicating a long name, such as "Pacific Standard Time."
static final int	SHORT	A style specifier for getDisplayName() indicating a short name, such as "PST."

Constructor Summary

Constructors

Constructor	Description
TimeZone()	Sole constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
static Stream<String>	availableIDs()	Returns a stream of the available IDs supported.
static Stream<String>	availableIDs(int rawOffset)	Gets the available IDs according to the given time zone offset in milliseconds.
Object	clone()	Creates a copy of this TimeZone.
static String[]	getAvailableIDs()	Returns an array of the available IDs supported.

static String[]	**getAvailableIDs**(int rawOffset)	Gets the available IDs according to the given time zone offset in milliseconds.
static TimeZone	**getDefault**()	Gets the default TimeZone of the Java virtual machine.
final String	**getDisplayName**()	Returns a long standard time name of this TimeZone suitable for presentation to the user in the default locale.
final String	**getDisplayName**(boolean daylight, int style)	Returns a name in the specified style of this TimeZone suitable for presentation to the user in the default locale.
String	**getDisplayName**(boolean daylight, int style, Locale locale)	Returns a name in the specified style of this TimeZone suitable for presentation to the user in the specified locale.
final String	**getDisplayName**(Locale locale)	Returns a long standard time name of this TimeZone suitable for presentation to the user in the specified locale.
int	**getDSTSavings**()	Returns the amount of time to be added to local standard time to get local wall clock time.
String	**getID**()	Gets the ID of this time zone.
abstract int	**getOffset**(int era, int year, int month,	Gets the time zone offset, for current date,

	int day, int dayOfWeek, int milliseconds)	modified in case of daylight savings.
int	**getOffset**(long date)	Returns the offset of this time zone from UTC at the specified date.
abstract int	**getRawOffset**()	Returns the amount of time in milliseconds to add to UTC to get standard time in this time zone.
static TimeZone	**getTimeZone**(String ID)	Gets the TimeZone for the given ID.
static TimeZone	**getTimeZone** (ZoneId zoneId)	Gets the TimeZone for the given zoneId.
boolean	**hasSameRules** (TimeZone other)	Returns true if this zone has the same rule and offset as another zone.
abstract boolean	**inDaylightTime** (Date date)	Queries if the given date is in Daylight Saving Time in this time zone.
boolean	**observesDaylightTime**()	Returns true if this TimeZone is currently in Daylight Saving Time, or if a transition from Standard Time to Daylight Saving Time occurs at any future time.
static void	**setDefault** (TimeZone zone)	Sets the TimeZone that is returned by the getDefault method.
void	**setID**(String ID)	Sets the time zone ID.
abstract void	**setRawOffset** (int offsetMillis)	Sets the base time zone offset to GMT.

ZoneId	**toZoneId**()	Converts this TimeZone object to a ZoneId.
abstract boolean	**useDaylightTime**()	Queries if this TimeZone uses Daylight Saving Time.

Methods declared in class Object

equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

SHORT

```
public static final int SHORT
```

A style specifier for getDisplayName() indicating a short name, such as "PST."

Since:

1.2

See Also:

LONG, Constant Field Values

LONG

```
public static final int LONG
```

A style specifier for getDisplayName() indicating a long name, such as "Pacific Standard Time."

Since:

1.2

See Also:

SHORT, Constant Field Values

Constructor Details

TimeZone

```
public TimeZone()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Details

getOffset

```
public abstract int getOffset(int era,
                              int year,
                              int month,
                              int day,
                              int dayOfWeek,
                              int milliseconds)
```

Gets the time zone offset, for current date, modified in case of daylight savings. This is the offset to add to UTC to get local time.

This method returns a historically correct offset if an underlying `TimeZone` implementation subclass supports historical Daylight Saving Time schedule and GMT offset changes.

Parameters:

`era` - the era of the given date.

`year` - the year in the given date.

`month` - the month in the given date. Month is 0-based. e.g., 0 for January.

`day` - the day-in-month of the given date.

`dayOfWeek` - the day-of-week of the given date.

`milliseconds` - the milliseconds in day in *standard* local time.

Returns:

the offset in milliseconds to add to GMT to get local time.

See Also:

`Calendar.ZONE_OFFSET`, `Calendar.DST_OFFSET`

getOffset

```
public int getOffset(long date)
```

Returns the offset of this time zone from UTC at the specified date. If Daylight Saving Time is in effect at the specified date, the offset value is adjusted with the amount of daylight saving.

This method returns a historically correct offset value if an underlying TimeZone implementation subclass supports historical Daylight Saving Time schedule and GMT offset changes.

Parameters:

`date` - the date represented in milliseconds since January 1, 1970 00:00:00 GMT

Returns:

the amount of time in milliseconds to add to UTC to get local time.

Since:

1.4

See Also:

`Calendar.ZONE_OFFSET`, `Calendar.DST_OFFSET`

setRawOffset

```
public abstract void setRawOffset(int offsetMillis)
```

Sets the base time zone offset to GMT. This is the offset to add to UTC to get local time.

If an underlying `TimeZone` implementation subclass supports historical GMT offset changes, the specified GMT offset is set as the latest GMT offset and the difference from the known latest GMT offset value is used to adjust all historical GMT offset values.

Parameters:

`offsetMillis` - the given base time zone offset to GMT.

getRawOffset

```
public abstract int getRawOffset()
```

Returns the amount of time in milliseconds to add to UTC to get standard time in this time zone. Because this value is not affected by daylight saving time, it is called *raw offset*.

If an underlying `TimeZone` implementation subclass supports historical GMT offset changes, the method returns the raw offset value of the current date. In Honolulu, for example, its raw offset changed from GMT-10:30 to GMT-10:00 in 1947, and this method always returns -36000000 milliseconds (i.e., -10 hours).

Returns:

the amount of raw offset time in milliseconds to add to UTC.

See Also:

`Calendar.ZONE_OFFSET`

getID

`public String getID()`

Gets the ID of this time zone.

Returns:

the ID of this time zone.

setID

`public void setID(String ID)`

Sets the time zone ID. This does not change any other data in the time zone object.

Implementation Requirements:

The default implementation throws a `NullPointerException` if `ID` is `null`

Parameters:

`ID` - the new time zone ID.

Throws:

`NullPointerException` - This method may throw a `NullPointerException` if `ID` is `null`

getDisplayName

`public final String getDisplayName()`

Returns a long standard time name of this `TimeZone` suitable for presentation to the user in the default locale.

This method is equivalent to:

```
getDisplayName(false, LONG,
               Locale.getDefault(Locale.Category.DISPLAY));
```

Returns:

the human-readable name of this time zone in the default locale.

Since:

1.2

See Also:

`getDisplayName(boolean, int, Locale)`,
`Locale.getDefault(Locale.Category)`,
`Locale.Category`

getDisplayName

```
public final String getDisplayName(Locale locale)
```

Returns a long standard time name of this `TimeZone` suitable for presentation to the user in the specified `locale`.

This method is equivalent to:

```
getDisplayName(false, LONG, locale);
```

Parameters:

`locale` - the locale in which to supply the display name.

Returns:

the human-readable name of this time zone in the given locale.

Throws:

`NullPointerException` - if `locale` is `null`.

Since:

1.2

See Also:

`getDisplayName(boolean, int, Locale)`

getDisplayName

```
public final String getDisplayName(boolean daylight,
                                   int style)
```

Returns a name in the specified `style` of this `TimeZone` suitable for presentation to the user in the default locale. If the specified `daylight` is `true`, a Daylight Saving Time name is returned (even if this `TimeZone` doesn't observe
Daylight Saving Time). Otherwise, a Standard Time name is returned.

This method is equivalent to:

```
getDisplayName(daylight, style,
               Locale.getDefault(Locale.Category.DISPLAY));
```

Parameters:

`daylight` - `true` specifying a Daylight Saving Time name, or `false` specifying a Standard Time name

`style` - either `LONG` or `SHORT`

Returns:

the human-readable name of this time zone in the default locale.

Throws:

`IllegalArgumentException` - if `style` is invalid.

Since:

1.2

See Also:

`getDisplayName(boolean, int, Locale)`,
`Locale.getDefault(Locale.Category)`,
`Locale.Category`,
`DateFormatSymbols.getZoneStrings()`

getDisplayName

```
public String getDisplayName(boolean daylight,
                             int style,
                             Locale locale)
```

Returns a name in the specified `style` of this `TimeZone` suitable for presentation to the user in the specified `locale`. If the specified `daylight` is `true`, a Daylight Saving Time name is returned (even if this `TimeZone` doesn't observe
Daylight Saving Time). Otherwise, a Standard Time name is returned.

When looking up a time zone name, the default `Locale` search path of `ResourceBundle` derived from the specified `locale` is used. (No fallback `Locale` search is performed.) If a time zone name in any `Locale` of the search path, including `Locale.ROOT`, is found, the name is returned. Otherwise, a string in the normalized custom ID format is returned.

Implementation Requirements:

The default implementation throws an `IllegalArgumentException` if `style` is invalid or a `NullPointerException` if `ID` is `null`.

Parameters:

`daylight` - `true` specifying a Daylight Saving Time name, or `false` specifying a Standard Time name

`style` - either `LONG` or `SHORT`

`locale` - the locale in which to supply the display name.

Returns:

the human-readable name of this time zone in the given locale.

Throws:

IllegalArgumentException - This method may throw an IllegalArgumentException if style is invalid.

NullPointerException - This method may throw a NullPointerException if ID is null

Since:

1.2

See Also:

DateFormatSymbols.getZoneStrings()

getDSTSavings

```
public int getDSTSavings()
```

Returns the amount of time to be added to local standard time to get local wall clock time.

The default implementation returns 3600000 milliseconds (i.e., one hour) if a call to useDaylightTime() returns true. Otherwise, 0 (zero) is returned.

If an underlying TimeZone implementation subclass supports historical and future Daylight Saving Time schedule changes, this method returns the amount of saving time of the last known Daylight Saving Time rule that can be a future prediction.

If the amount of saving time at any given time stamp is required, construct a Calendar with this TimeZone and the time stamp, and call Calendar.get(Calendar.DST_OFFSET).

Returns:

the amount of saving time in milliseconds

Since:

1.4

See Also:

inDaylightTime(Date),
getOffset(long),
getOffset(int,int,int,int,int,int),
Calendar.ZONE_OFFSET

useDaylightTime

```
public abstract boolean useDaylightTime()
```

Queries if this TimeZone uses Daylight Saving Time.

If an underlying TimeZone implementation subclass supports historical and future Daylight Saving Time schedule changes, this method refers to the last known Daylight Saving Time rule that can be a future prediction and may not be the same as the current rule. Consider calling observesDaylightTime() if the current rule should also be taken into account.

Returns:

true if this TimeZone uses Daylight Saving Time, false, otherwise.

See Also:

inDaylightTime(Date), Calendar.DST_OFFSET

observesDaylightTime

```
public boolean observesDaylightTime()
```

Returns true if this TimeZone is currently in Daylight Saving Time, or if a transition from Standard Time to Daylight Saving Time occurs at any future time.

The default implementation returns true if useDaylightTime() or inDaylightTime(new Date()) returns true.

Returns:

true if this TimeZone is currently in Daylight Saving Time, or if a transition from Standard Time to Daylight Saving Time occurs at any future time; false otherwise.

Since:

1.7

See Also:

useDaylightTime(), inDaylightTime(Date), Calendar.DST_OFFSET

inDaylightTime

```
public abstract boolean inDaylightTime(Date date)
```

Queries if the given date is in Daylight Saving Time in this time zone.

Parameters:

date - the given Date.

Returns:

true if the given date is in Daylight Saving Time, false, otherwise.

Throws:

NullPointerException - This method may throw a NullPointerException if date is null

getTimeZone

```
public static TimeZone getTimeZone(String ID)
```

Gets the TimeZone for the given ID.

Parameters:

ID - the ID for a TimeZone, either an abbreviation such as "PST", a full name such as "America/Los_Angeles", or a custom ID such as "GMT-8:00". Note that the support of abbreviations is for JDK 1.1.x compatibility only and full names should be used.

Returns:

the specified TimeZone, or the GMT zone if the given ID cannot be understood.

Throws:

NullPointerException - if ID is null

getTimeZone

```
public static TimeZone getTimeZone(ZoneId zoneId)
```

Gets the TimeZone for the given zoneId.

Parameters:

zoneId - a ZoneId from which the time zone ID is obtained

Returns:

the specified TimeZone, or the GMT zone if the given ID cannot be understood.

Throws:

NullPointerException - if zoneId is null

Since:

1.8

toZoneId

```
public ZoneId toZoneId()
```

Converts this TimeZone object to a ZoneId.

Returns:

a ZoneId representing the same time zone as this TimeZone

Since:

1.8

getAvailableIDs

```
public static String[] getAvailableIDs(int rawOffset)
```

Gets the available IDs according to the given time zone offset in milliseconds.

API Note:

Consider using `availableIDs(int)` which returns a stream of the available time zone IDs according to the given offset.

Parameters:

`rawOffset` - the given time zone GMT offset in milliseconds.

Returns:

an array of IDs, where the time zone for that ID has the specified GMT offset. For example, "America/Phoenix" and "America/Denver" both have GMT-07:00, but differ in daylight saving behavior.

See Also:

`getRawOffset()`, `availableIDs(int)`

getAvailableIDs

```
public static String[] getAvailableIDs()
```

Returns an array of the available IDs supported.

API Note:

Consider using `availableIDs()` which returns a stream of the available time zone IDs.

Returns:

an array of the available IDs supported

See Also:

`availableIDs()`

availableIDs

```
public static Stream<String> availableIDs(int rawOffset)
```

Gets the available IDs according to the given time zone offset in milliseconds.

Implementation Note:

Unlike `getAvailableIDs(int)`, this method does not create a copy of the `TimeZone` IDs array.

Parameters:

`rawOffset` - the given time zone GMT offset in milliseconds.

Returns:

a stream of IDs, where the time zone for that ID has the specified GMT offset. For example, "America/Phoenix" and "America/Denver" both have GMT-07:00, but differ in daylight saving behavior.

Since:

25

See Also:

`getRawOffset()`, `getAvailableIDs(int)`

availableIDs

```
public static Stream<String> availableIDs()
```

Returns a stream of the available IDs supported.

Implementation Note:

Unlike `getAvailableIDs()`, this method does not create a copy of the `TimeZone` IDs array.

Returns:

a stream of the available IDs supported

Since:

25

See Also:

`getAvailableIDs()`

getDefault

```
public static TimeZone getDefault()
```

Gets the default `TimeZone` of the Java virtual machine. If the cached default `TimeZone` is available, its clone is returned. Otherwise, the method takes the following steps to determine the default time zone.

- Use the `user.timezone` property value as the default time zone ID if it's available.
- Detect the platform time zone ID. The source of the platform time zone and ID mapping may vary with implementation.
- Use `GMT` as the last resort if the given or detected time zone ID is unknown.

The default `TimeZone` created from the ID is cached, and its clone is returned. The `user.timezone` property value is set to the ID upon return.

Returns:

the default `TimeZone`

See Also:

`setDefault(TimeZone)`

setDefault

`public static void setDefault(TimeZone zone)`

Sets the `TimeZone` that is returned by the `getDefault` method. `zone` is cached. If `zone` is null, the cached default `TimeZone` is cleared. This method doesn't change the value of the `user.timezone` property.

Parameters:

`zone` - the new default `TimeZone`, or null

See Also:

`getDefault()`

hasSameRules

`public boolean hasSameRules(TimeZone other)`

Returns true if this zone has the same rule and offset as another zone. That is, if this zone differs only in ID, if at all. Returns false if the other zone is null.

Parameters:

`other` - the `TimeZone` object to be compared with

Returns:

true if the other zone is not null and is the same as this one, with the possible exception of the ID

Since:

1.2

clone

`public Object clone()`

Creates a copy of this `TimeZone`.

Overrides:

`clone` in class `Object`

Returns:

a clone of this `TimeZone`

See Also:

`Cloneable`

Report a bug or suggest an enhancement

For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.